

Pruebas Automatizadas — Control de EPP

Aplicación: [control-epp-ht-production.up.railway.app](#) · Documento relacionado: [HT-AP-02](#)

1 Propósito

Describir la infraestructura de pruebas automatizadas de la aplicación: qué se verifica, cómo ejecutarla y cómo mantenerla al agregar nuevas funcionalidades.

El objetivo es evitar regresiones, en particular los desajustes entre la forma de datos que entrega el backend y la que espera el frontend, que fueron la causa de los crashes corregidos en esta fase.

2 Alcance

2.1 Qué se prueba

Backend — pruebas de integración (12 casos). Levantan el servidor Express contra una base de datos SQLite temporal y verifican la forma de respuesta de cada endpoint contra lo que el frontend consume:

- Autenticación (login de administrador).
- GET /users, GET /epp, GET /stock (incluye alias epp_id / epp_nombre).
- Creación de EPP e ingreso de stock.
- Entrega directa (POST /entregas/directa) sin solicitud previa.
- Movimientos de stock ({epp, movimientos} con usuario_nombre).
- Asignaciones por trabajador (asignaciones + historial_entregas).
- Matriz de asignaciones (devuelta como array).
- Reportes: stock crítico ({total, items}) y vencimientos.
- Solicitud completa: crear, listar pendientes con items_count, ver detalle con stock_disponible.
- Eliminación de EPP: 409 si tiene registros asociados, 200 si no.

Frontend — pruebas de componente (22 casos, 20/20 páginas). Renderizan cada página con el módulo api simulado (mock) y verifican que rendericen correctamente y no crasheen con la forma de datos real, incluyendo casos de datos vacíos.

2.2 Qué NO se prueba (limitaciones conocidas)

- Flujos end-to-end reales (navegador + backend + base de datos juntos).
- Generación y contenido de los PDF (services/pdf.js).
- Envío real de correos (el servicio falla en silencio sin SMTP).
- Estilos visuales / layout (CSS).
- Carga y almacenamiento físico de archivos en el volumen de Railway.

3 Cómo ejecutar las pruebas

3.1 Backend

```
cd backend
npm install      # solo la primera vez
npm test
```

Usa el runner nativo de Node (node --test), sin dependencias adicionales. Cada corrida crea una base de datos temporal aislada; no toca datos reales.

3.2 Frontend

```
cd frontend
npm install      # solo la primera vez
npm test
```

Usa Vitest + React Testing Library sobre un entorno jsdom. Estas dependencias son de desarrollo y no afectan el despliegue en Railway.

3.3 Requisitos

- Node.js 18 o superior (validado en Node 22).
- No requiere base de datos, SMTP ni red.

4 Estructura de archivos

```
backend/
  server.js          # exporta { app, initDb }; escucha solo si se ejecuta directo
  test/
    api.test.js      # 12 pruebas de integración de la API

frontend/
  vite.config.js     # bloque "test" (jsdom, setup)
  src/test/
    setup.js         # matchers jest-dom + limpieza entre pruebas
    utils.jsx        # helper renderPage (Router + AuthProvider + localStorage)
  src/pages/**/*.*.test.jsx # una prueba co-ubicada por cada página
```

5 Cómo agregar una prueba nueva

5.1 Al agregar un endpoint en el backend

- Abrir backend/test/api.test.js.
- Agregar un bloque test('...', async () => { ... }).
- Usar el helper req(method, path, { body, form }); el token de admin se obtiene en test.before.
- Afirmary la forma exacta de la respuesta (campos y tipos) que el frontend espera.

5.2 Al agregar una página en el frontend

- Crear NombrePagina.test.jsx junto al componente (mismo directorio), para que el mock coincida con el import.
- Simular las respuestas de api con vi.fn() / mockResolvedValue.
- Renderizar con renderPage(<NombrePagina />, { user, path, route }) cuando use useAuth o useParams.
- Afirmary que el dato clave aparece en pantalla (y, si corresponde, un caso con datos vacíos).

5.3 Regla de oro

Cada vez que el frontend consume un endpoint nuevo o modifique uno existente, verificar que la forma de datos (array vs objeto, nombres de campos) coincida en ambos lados y agregar/ajustar la prueba correspondiente.

6 Resultado actual

Suite	Casos	Estado
Backend	12	verde
Frontend	22	verde

Última verificación:
2026-06-14.

